MSc Computer Network Administration and Management

INFRASTRUCTURE AUTOMATION

LAMP STACK AUTO-DEPLOYMENT USING TERRAFORM AND ANSIBLE

Masters Project (COMP) (SEP 2024) - M32616

Supervisor : Mr Ioannis Kagalidis
Student Name: Sriram Arumugam
Module Submission Date : 17/09/2025

Student Number						
2	2	9	0	3	4	5

Table of Contents

Abstract

Acknowledgment

1. Chapter 1: Introduction ---------- 6
   1.1 Aim of the Project ---------- 6
   1.2 Objective of the Project ---------- 7
   1.3 Methodology ---------- 7
   1. Commands Used---------- 8

2. Chapter 2: Literature Review----------10
   2.1 Infrastructure as Code (IaC) ----------10
   2.2 Ansible with Terraform---------- 11
   2.3 Cloud Computing ----------12
   2.4 Secure Shell (SSH) ---------- 13
   2.5 Conclusion ---------- 14
3. Chapter 3: Project Design and Architecture ----------15
   3.1 Overview of the Design ----------15
   3.2 Project Components ----------15
   3.3 Architecture Flow ---------- 16
   3.4 Key Design Considerations ---------- 17
   3.5 Flowchart: Project Design and Architecture ---------- 18
   3.6 Explanation of Flowchart ---------- 19
   3.7 Conclusion ---------- 19

4. Chapter 4: Tools Used ---------- 20
   4.1 Ansible ---------- 20
   4.2 Terraform ---------- 21
   4.3 AWS (Amazon Web Services) ---------- 21
   4.4 MySQL ---------- 22
   4.5 Linux/Ubuntu ---------- 23
   4.6 Apache ---------- 23
   4.7 PHP ---------- 23
   4.8 VS Code / Terminal ---------- 24
   4.9 Integration of Tools ---------- 24

5. Chapter 5: Terraform Implementation ---------- 25
5.1 Terraform Workflow and Execution ---------- 26
5.2 Error Handling and Troubleshooting ---------- 27
5.3 Security Issues ---------- 27
5.4 Terraform's Role in the Project ---------- 28
5.5 Conclusion ---------- 28

6. Chapter 6: Ansible Implementation ---------- 29
6.1 Introduction ---------- 29
6.2 Ansible Inventory and Playbooks ---------- 30
6.3 Running the Ansible Playbook ---------- 32
6.4 Error Handling and Troubleshooting ---------- 33
6.5 Ansible's Role in the Project ---------- 33
6.6 Conclusion ---------- 34

7. Chapter 7: Integration and Results ---------- 35
7.1 Integration of Terraform and Ansible ---------- 36
7.2 Challenges Faced During Integration ---------- 36
7.3 Results of Integration ---------- 37
7.4 Testing and Verification ---------- 38
7.5 Conclusion ---------- 39

8. Chapter 8: Challenges and Learnings ---------- 40
8.1 Choosing the Correct EC2 Instance Type ---------- 40
8.2 AWS IAM Configuration and Access Issues ---------- 41
8.3 Installing and Configuring Tools ---------- 42
8.4 Challenges in Connectivity and SSH Access ---------- 43
8.5 Rebooting and Virtual Machine Management ---------- 44
8.6 Idempotency and Re-running Ansible Playbooks ---------- 45
8.7 Security Issues and Configuration Management ---------- 46
8.8 Conclusion ---------- 47

9. Chapter 9: Screenshots & Testing Evidence ---------- 48
9.1 Terraform Execution and EC2 Instance Provisioning ---------- 48
9.2 Ansible Playbook Execution ---------- 49
9.3 AWS EC2 Instance Page ---------- 49
9.4 AWS Security Group Configuration ---------- 50
9.5 RSA Key Pair (.pem) Configuration ---------- 50

10. Chapter 10: Conclusion & Future Improvements ---------- 51
10.1 Conclusion ---------- 52
10.2 Future Improvements ---------- 54
10.3 Conclusion ---------- 55
11. References ----------56
12. Appendix ----------58
13.

Abstract

This project focuses on **Infrastructure Automation** through the deployment of a **LAMP stack** (Linux, Apache, MySQL, PHP) on **AWS EC2 instances** using **Terraform** and **Ansible**. The primary goal was to automate the entire process of provisioning cloud infrastructure and configuring the LAMP stack, reducing manual setup and ensuring a **consistent**, **reliable**, and **scalable** deployment solution. **Terraform** was used to provision the necessary AWS resources, including EC2 instances and security groups, while **Ansible** was employed to automate the installation and configuration of the LAMP stack on the provisioned instances. The project demonstrated the power of **Infrastructure as Code (IaC)** in **cloud environments**, emphasizing the efficiency of automation in a **DevOps**-driven approach. Through the integration of these tools, the project enabled **one-click** deployment, ensuring that the LAMP stack can be quickly replicated and scaled in different environments. This solution is beneficial for developers, IT administrators, and organizations seeking to reduce deployment times and improve consistency across environments. The project also explores potential **future improvements**, including enhanced security, scalability, CI/CD integration, and cost optimization strategies.

Acknowledgement

I would like to express my sincere gratitude to everyone who contributed to the successful completion of this project. Firstly, I would like to thank my supervisor, **Mr Ioannis Kagalidis**, for their continuous support, guidance, and valuable feedback throughout this project. Their insights were pivotal in shaping the direction of the project and helping me overcome various challenges along the way.

I would also like to extend my thanks to the **University of Portsmouth** for providing the resources and learning environment that made this project possible. The tools and platforms available have been instrumental in executing the project successfully. I've done all my projects on the big screen in the University Library.

A special thanks to my **family and friends**, whose constant encouragement and support have motivated me during difficult times, especially when dealing with the complexities of this project.

Lastly, I would like to acknowledge the **open-source community**, whose tools **Terraform**, **Ansible**, and **AWS** have significantly contributed to the ease of implementing this project. Without these tools, the automation and deployment processes would have been far more challenging.

Chapter 1: INTRODUCTION

In the world of cloud computing has transformed the best way for businesses to manage their infrastructure and applications. One of the most crucial areas in cloud management is the application deployment, which traditionally requires significant manual intervention. In particular, the LAMP stack (Linux, Apache, MySQL, PHP) is one of the most widely used stacks for developing and hosting web applications. However, deploying and maintaining LAMP stacks across multiple servers manually can be time-consuming and difficult to scale. The manual process introduces inconsistencies in environments and can lead to mostly human errors that disrupt the deployment process.

To overcome these challenges, automation is the best thing which can be repeatable without any errors. For this LAMP stack we use automation tools like Terraform and Ansible to offer a solution for deploying infrastructure and applications in a repeatable, efficient, and consistent manner. This project aims to automate the deployment of a LAMP stack on AWS EC2 instances using Terraform for infrastructure provisioning and Ansible for configuration management. By leveraging these tools, the project aims to improve deployment speed, consistency, and scalability while minimizing human error is our AIM of our project.

1.1 Aim of the Project

The Aim of the project is to automate the deployment of a LAMP stack (Linux, Apache, MySQL, PHP) on AWS EC2 instances using Terraform to provision and manage infrastructure in a cloud environment. This goal emphasizes automation which is ease of scaling, and deployment consistency without errors, which will lead us to create an efficient, cost-effective solution for hosting dynamic web applications by leveraging AWS Free Tier and Terraform for reproducibility and scalability.

To summarise the project in one line “Automating the full deployment of a LAMP server on AWS using Terraform for infrastructure and Ansible for configuration which all be reproducible with a single command”

1.2 Objective of the Project:

The Primary Objective of this Project is:

1. **Provisioning Infrastructure**: Use Terraform to automatically provision the required AWS infrastructure (EC2 instance, security group, etc.).
2. **Installing LAMP Stack**: Install Apache, MySQL, and PHP on the EC2 instance to create a fully functional web server environment
3. **Testing and Validation**: Ensure the LAMP stack works by testing Apache, MySQL, and PHP functionality, including web page access and database connectivity.
4. **Security and Access Control**: Secure the EC2 instance using firewall rules (security groups), SSH access configurations, and MySQL user privileges.
5. **Documentation and Reporting**: Provide a comprehensive report and documentation on the deployment process, configurations, and the use of Terraform.

This report will explore the process of automating the deployment of the LAMP stack, discussing the tools used, implementation details, challenges faced, and the results obtained. Additionally, future improvements and potential use cases of the system will be discussed, providing a comprehensive overview of the project's impact and relevance in the field of cloud computing and DevOps which will help the core users who will use it to focus more on developing applications.

1.3 Methodology

The methodology employed in this project successfully automated the process of provisioning cloud infrastructure and configuring software for a LAMP stack. By using Terraform for infrastructure provisioning and Ansible for configuration management, the project demonstrated how modern DevOps practices can streamline cloud infrastructure deployment. Through integration, testing, and optimization, the project created a repeatable and scalable solution, making it easier to deploy web applications without manual intervention. The tools were integrated to ensure efficiency and consistency, reducing human error and making the deployment process more reliable.

1.4 Commands used:

In this project, **Linux commands** play a vital role in configuring and managing services such as **Ansible** and **AWS**. These commands provide fine-grained control over tool installations, configurations, and testing, facilitating smooth interaction with the system and ensuring seamless execution of tasks throughout the project.

Basic Commands:

man: Displays detailed information about Linux commands.

cp: Used to copy files from one location to another.

mv: Moves files or directories from one location to another.

cd: Changes the current working directory.

./: Executes a file or script in current Directory.

dnf: A command to install and manage packages (download).

yum: A package manager command used for downloading and updating software.

ping: Sends network packets to check the connectivity status between systems.

telnet: Tests network connectivity by establishing a remote connection.

nslookup: Troubleshoots network issues by querying DNS servers.

systemctl: Controls and manages the state of services on the system.

chmod: Modifies the permissions of files or directories.

mkdir: Creates a new directory.

chown: Changes the owner of files or directories.

echo: Displays a message or value, often used to test web page output or display variables.

Terraform Commands:

#terraform **init** => Initializes the Terraform working directory, downloads provider plugins, and sets up the backend

#terraform **plan** => Generates an execution plan, showing what actions Terraform will take to apply the configuration

#terraform **apply** => Applies the execution plan, provisioning or updating resources in AWS as defined in the Terraform configuration files

#terraform **destroy** => Destroys the infrastructure created by Terraform, allowing for resource clean-up

#terraform **validate** => Validates the syntax and configuration of Terraform files, ensuring that they are correct before applying them (Like a local check)

#terraform **show** => Displays the current state of the infrastructure managed by Terraform

#terraform **output** => Retrieves the outputs defined in the Terraform configuration (e.g., EC2 instance public IP).

Ansible Commands:

#ansible-playbook filename.yml => To run the Ansible Playbook

#ansible-playbook -m ping all => Uses Ansible's ping module to check if the target machines like the EC2 instances are reachable.

#ansible-playbook –check filename.yml => Runs the playbook and shows changes that will be made to the files and allows us to review modifications before applying them.

#ansible-playbook -m setup all => Gather all Facts from the targeting machine.

SSH Commands:

#ssh -i mykey.pem user@0.0.0.0 (Our Public IP) => Connects EC2 Instance using Private key (mykey.pem) by SSH Protocol.

#sudo systemctl start apache2 => Start the Apache Web server on the EC2 Instances

#sudo systemctl enable apache2 => Ensures the Apache Web server is on Boot automatically.

#sudo systemctl Status apache2 => Displays the Apache Web server current status (Active/Stopped)

Chapter 2 : Literature Review

This Chapter speaks about the key concepts and technologies used in the project, providing the necessary academic context and insights as a guidance of literature which I get to learn from them as a mentor. As I come to know the term "mentor" originates from Greek mythology. In Homer's *Odyssey*, Mentor was a trusted friend of Odysseus, tasked with guiding and advising his son, Telemachus. Mentor's role was to offer wisdom and support, helping Telemachus navigate challenges. Over time, the name "Mentor" became synonymous with someone who provides guidance and advice, especially in personal or professional contexts. Today, a mentor is understood as a trusted advisor who shares knowledge and experience, helping others grow and succeed in their personal or career development. As my Supervisor is my mentor of this project who uplifts me when I face a hurdle. Some of the basic understanding of the project will be covered in this Chapter.

2.1 IAC (Infrastructure as Code) :

In the ever-evolving landscape of cloud computing and on-premise infrastructure, businesses face the challenge of efficiently managing their resources across various platforms and architectures. This is where Terraform, an innovative infrastructure as code tool, emerges as a logical and powerful choice. Employing a declarative approach and utilizing its own language, the HashiCorp Configuration Language (HCL), Terraform offers unparalleled simplicity and readability for humans, making it accessible to both seasoned professionals and newcomers (Abbas Mehdi, 2023). Several tools support the concept of IAC, including Terraform, CloudFormation, and Ansible. These tools allow teams to define infrastructure in configuration files, which can be executed to automatically provision resources in cloud environments like AWS.

To minimize errors and ensure consistency, it is possible to employ mathematical models to define and specify the infrastructure configuration, including network setup, server configurations, resource allocation, and other related parameters. With these models, the dependencies and relationships between components can be captured, making them more efficient.(Milandeep Kour Bali, 2023). In this project, Terraform is used as the primary tool for implementing **IAC**, which automates the process of provisioning AWS EC2 instances, security groups, and related infrastructure components. This ensures a consistent and scalable environment for deploying the LAMP stack.

2.2 Ansible with Terraform:

Ansible is an open-source automation tool that automates configuration management, application deployment, and task orchestration. It uses playbooks (written in YAML) to define tasks that need to be executed on remote machines while Terraform is an open-source tool used for Infrastructure as Code (IAC). It allows users to define and provision infrastructure resources (e.g., EC2 instances, networks, and databases) using configuration files written in HCL (HashiCorp Configuration Language) which they provide certifications for the proof of skill.Combining Terraform and Ansible allows for a comprehensive automation solution, with Terraform handling infrastructure provisioning and Ansible managing configuration.

In this project, the combination of the **Terraform** for provisioning the AWS infrastructure and **Ansible** for configuring the LAMP stack ensures that the entire deployment process is automated and easily reproducible.Server automation has become crucial in systems administration, especially with the dynamic nature of modern application environments. Tools like **Ansible** are commonly employed to simplify the server setup process by implementing standardized procedures for provisioning new servers. This automation not only ensures consistency but also minimizes the potential for human error that often occurs during manual configurations(Erika Heidi , 2019).

Terraform and Ansible are two powerful tools that complement each other in the automation of infrastructure and configuration, making them essential for LAMP stack deployment. Both use declarative configuration, allowing users to define the desired state for their systems or infrastructure. While Ansible is agentless, simplifying the setup by not requiring agents on target machines, it excels in automating the configuration and management of the LAMP stack components such as Apache, MySQL, and PHP. On the other hand, Terraform specializes in multi-cloud support, enabling the provisioning and management of infrastructure across various cloud providers like AWS, Google Cloud, and Azure. It also features state management, which allows users to track and maintain infrastructure changes over time. Together, Terraform's infrastructure provisioning and Ansible's configuration management form a seamless and integrated solution, ensuring a consistent, scalable, and automated LAMP stack deployment that cannot be separated in achieving an efficient setup.

2.3 Cloud Computing

In simple terms Cloud computing refers to the delivery of computing services over the internet. Basically a storage space for everything but not as local storage but as fast as local storage respective to the Internet speed. In Cloud Environment Internet is the virtual SATA cable for our connecting medium. Cloud providers like AWS, Google Cloud, and Microsoft Azure offer on-demand access to these resources, enabling businesses to scale their operations quickly without the need for physical infrastructure. Cloud services can be used to develop and distribute web applications, host computing infrastructure for businesses, store digital media, provide browser-based software, and process data workload(Erin Glass ,2020). This project utilizes AWS EC2 instances to provision the infrastructure required to host the LAMP stack.

2.4 Secure Shell (SSH)

SSH is a crucial protocol for securely managing and administering remote systems over insecure networks like the internet. In the context of my project, which involves automating the deployment of a LAMP stack on AWS EC2 instances, SSH plays a pivotal role. It enables secure remote access to the EC2 instances where the LAMP stack is installed. By using SSH, I can securely connect from my local Ubuntu system to the EC2 instances, allowing me to execute commands, transfer files, and manage the configuration of the LAMP stack remotely, all while ensuring the security of the data transmitted between the two systems. The data encryption provided by SSH ensures that any interaction with the EC2 instances, such as file transfers or command execution, is protected from potential security threats on an open network.

SSH not only facilitates secure remote access but also supports secure file transfers between my local system and the EC2 instances. During the LAMP stack automation process, I use SSH to transfer configuration files, scripts, and other necessary files from my local Ubuntu machine to the AWS EC2 instances. This allows for efficient configuration management and seamless integration of the Apache, MySQL, and PHP components of the LAMP stack. Moreover, with SSH's public key authentication, I ensure that only authorized users can access the EC2 instances, adding an additional layer of security to the infrastructure. Overall, SSH plays a vital role in enabling secure, remote management of the LAMP stack on AWS, allowing me to perform tasks such as installation, configuration, and maintenance without compromising on security.

The Key thing we worked on so far from setting the Virtual machine to Creating Instances in AWS and setting up the terraform file until everything's sorted out and mainly our work is to set the basic commands which will deploy in the required machine at this deployment stage comes the SSH so we can manage the remote systems.SSH offers a relaxed transfer of records between the two nodes. Via encrypting the statistics in transit, attackers cannot get entry to the information being transferred. It offers a further layer of protection, which protects the facts from being compromised. (Rajat Saini, 2023).

2.5 Conclusion:

This literature review has provided an overview of the key concepts and tools used in this project, including Infrastructure as Code (IAC), the LAMP stack, and the integration of Ansible and Terraform for automation. We have also discussed how cloud computing works and the benefits of automation, which directly align with the goals of this project—automating LAMP stack deployment for efficiency, consistency, and scalability. The following sections of the report will delve deeper into the implementation details, challenges faced, and results obtained from applying these technologies.

Chapter 3: Project Design and Architecture

3.1 Overview of the Design

The architecture of the LAMP stack automation project focuses on Infrastructure Provisioning, Configuration Management and Automation Integration. By using **Terraform** to provision the necessary cloud resources (EC2, VPC, Security Groups) and install and configure the **LAMP stack** (Linux, Apache, MySQL, PHP) on the provisioned EC2 instance using Ansible and Combining them into a seamless automation process, where Terraform provisions resources and Ansible configures them.This architecture ensures that the entire process, from resource creation to application setup, is automated and repeatable with no errors.

3.**2. Project Components**

In this project, Terraform is primarily used for provisioning the infrastructure on AWS, ensuring that the cloud environment is properly set up before the LAMP stack is deployed. Using Terraform configuration files(e.g., main.tf, outputs.tf), we define the required resources in a declarative manner, including EC2 instances, networking, and security groups. These security groups are configured to allow necessary ports like 22 for SSH, 80 for HTTP, and 443 for HTTPS to facilitate secure communication with the EC2 instances. By managing the infrastructure as code (IaC), Terraform enables efficient and repeatable provisioning of resources, which is crucial for maintaining consistency across deployments.

On the configuration management side, Ansible is used to automate the setup of the LAMP stack on the provisioned EC2 instances. Through Ansible playbooks, the necessary software such as Apache, MySQL (or MariaDB), and PHP is installed and configured on the instances. The playbooks define the specific tasks to be performed, ensuring that the entire stack is correctly set up and ready for use. AWS services such as EC2 instances, VPC, and IAM roles are also leveraged to provide a secure and isolated network environment. The EC2 instances host the LAMP stack, while the VPC and subnet provide network isolation, and IAM roles ensure that proper permissions are in place for securely interacting with AWS services**.**

3.3 Architecture Flow:

The Terraform execution process begins with provisioning the EC2 instance using the configuration defined in the main.tf file. It then configures the necessary networking resources, including security groups and VPC settings, to ensure secure communication and proper isolation for the instance. After the EC2 instance is provisioned, Terraform outputs the instance's public IP, which is subsequently used in the Ansible execution process for configuring the instance. Once the EC2 instance is up and running, Ansible connects to the instance using the SSH and executes the playbook (lamp.yml). This playbook installs and configures the LAMP stack, including Apache, MySQL, and PHP, on the instance. The Apache server is set up to run PHP scripts, while MySQL is configured as the database server. The integration between Terraform and Ansible creates a fully automated, end-to-end solution. After Terraform provisions the infrastructure, Ansible automatically configures the LAMP stack, ensuring that the web server is fully functional in one seamless operation.

3.4 Key Design Considerations

Scalability:

The design is highly scalable, allowing for easy expansion of the environment. With Terraform, adding additional EC2 instances or scaling resources is a simple process. Moreover, Ansible can be adapted to handle multiple instances, making it easy to scale the setup as required.

Security:

Security is a top priority, with the use of security groups to limit access to only the necessary ports, protecting the EC2 instances from unauthorized access. Additionally, IAM roles and policies ensure that only authorized resources are able to interact with the EC2 instances, further strengthening the security framework.

Cost-Effectiveness:

By leveraging the AWS Free Tier and automating the deployment process, this project keeps costs low, making it a cost-effective solution, particularly for developers and small businesses looking for efficient cloud infrastructure without the high costs.

Reproducibility:

The use of Terraform scripts and Ansible playbooks ensures that the entire setup process is easily reproducible across different AWS accounts or environments. This guarantees a consistent and repeatable deployment process, streamlining the setup for future use.

3.5 Flowchart: Project Design and Architecture

Terraform Initialization

↓

Provision EC2 Instances

↓

Output EC2 Instance Public IP

↓

Ansible Playbook Execution

↓

Configure Apache to Run PHP

↓

Verify Apache and PHP

↓

Fully Automated Deployment

3.6 Explanation of Flowchart:

The flowchart begins with Terraform Initialization & Application, where Terraform initializes the configuration, applies it to provision the EC2 instance, and outputs the instance's public IP. This IP is then used by Ansible Playbook Execution, which connects to the EC2 instance and runs the lamp.yml playbook to install and configure Apache, MySQL, and PHP. Finally, Resulting in a Configured LAMP Stack, once the Ansible playbook finishes execution, the EC2 instance is fully set up with a working LAMP stack, making it ready for use.

3.7 Conclusion

This project demonstrates the power of automation in infrastructure and application deployment using Terraform and Ansible. The design aims to simplify and streamline the process of deploying a LAMP stack on AWS by automating both the infrastructure provisioning and configuration management. Terraform is used to initialize the environment, provision the EC2 instances, and configure essential networking components like security groups and VPC settings. It also ensures consistency by outputting the EC2 instance's public IP, which is then used by Ansible to configure the LAMP stack.

Ansible automates the installation and configuration of Apache, MySQL, and PHP on the EC2 instance, ensuring a fully functional LAMP stack with minimal manual intervention. This automation removes human errors, reduces setup time, and ensures a consistent, repeatable environment. The seamless integration of Terraform and Ansible provides a one-click deployment solution, making it easy to replicate the setup across different environments or AWS accounts.

Overall, the project highlights the advantages of Infrastructure as Code (IaC) and configuration management tools, creating a robust, scalable, and secure deployment process. This approach is ideal for developers and businesses seeking an efficient, automated solution for deploying and managing web applications in the cloud.

Chapter: 4 Tools Used :

In this project, several tools and technologies have been utilized to automate the deployment of the LAMP stack on AWS EC2 instances. These tools include Terraform, Ansible, AWS, MySQL, Linux/Ubuntu, Apache, PHP, and development tools like VS Code for the Terminal. Each of these tools plays a crucial role in provisioning, configuring, and managing the infrastructure, ensuring the project is automated, scalable, and repeatable. This Chapter explains how the tools are used and how it helps in the step by step process of this automation project.

4.1 Ansible

Ansible is an open-source automation tool that plays a central role in this project, specifically for configuring the LAMP stack on the EC2 instances. Ansible uses simple code (playbooks) to automate these tasks, making system management faster, easier, and more consistent without needing extra software agents.Ansible automates the installation of Apache, MySQL, and PHP the core components of the LAMP stack—by executing the playbook lamp.yml. The playbook defines tasks such as installing the necessary software packages, starting services, and configuring them to work together.

Ansible's idempotent nature ensures that if the playbook is run multiple times, the system remains consistent without redundant configurations. The inventory file used by Ansible connects to the AWS EC2 instances provisioned by Terraform, allowing the seamless automation of software installation and setup, ensuring that the LAMP stack is correctly configured on the cloud instances. By utilizing Ansible, the project reduces the manual effort involved in configuring the server and guarantees a consistent setup across different EC2 instances, making it ideal for both development and production environments.

(NOTE : **Idempotent** refers to a property where performing an operation multiple times has the same effect as performing it once.)

“This is the core of our project, where no errors occur as long as the pre-written scripts are properly configured for the environment, enabling seamless cloning without any issues.”

4.2 Terraform

Terraform is the OpenSource tool invented by HashiCorp, a California based Software Development company that allows us to define and manage Cloud Infrastructure. For this project it is a core tool used for provisioning the cloud infrastructure on AWS. It allows us to define the desired state of the infrastructure using code, which can be stored in version control and reused across different environments. In this project, Terraform is responsible for provisioning the EC2 instances, security groups, and Elastic IPs on AWS. Terraform's configuration files, primarily main.tf, define the AWS provider and the resources that need to be provisioned. The terraform init command initializes the working directory, while terraform apply creates the resources as defined.

By using Terraform, the project achieves infrastructure as code (IAC), which means that the entire AWS infrastructure can be replicated and version-controlled. Terraform interacts directly with AWS, ensuring that the EC2 instances are secure, scalable, and reproducible. This enables automation and a streamlined workflow for provisioning cloud resources.

4.3 **AWS (Amazon Web Services)**

This is an exciting open-source project, perfect for learning. However, as we used all open-source tools, this is the one area where we actually pay for the service. While the service is free for students to use for a limited time, in the real world, it becomes a paid service. This highlights the platform's service-based nature, where ongoing usage after the trial period incurs costs.AWS serves as the cloud platform hosting the entire infrastructure. In this project, AWS EC2 instances provide the computing resources where the LAMP stack is deployed. The instances are provisioned through Terraform and are automatically configured using Ansible.

AWS services like Security Groups and Elastic IPs ensure that the EC2 instances are secure and accessible over the internet. Security groups are defined to allow necessary ports (22 for SSH, 80 for HTTP, and 443 for HTTPS), ensuring that the EC2 instances can communicate securely with the outside world while preventing unauthorized access.AWS's scalability is also a key feature of this project, allowing the infrastructure to grow as needed by provisioning additional EC2 instances or scaling the existing instances, which aligns with the automation and repeatability goals of the project.

4.4. MySQL

SQL (Structured Query Language) is a programming language used to manage and interact with databases. It allows users to insert, update, delete, and query data efficiently, enabling communication between applications and databases. MySQL is the relational database system used to store data for the web applications hosted on the LAMP stack. In this project, Ansible is used to install MySQL (or MariaDB) on the EC2 instances and configure it to be accessible by PHP applications running on Apache.

The MySQL installation is automated through the Ansible playbook, ensuring that the database is correctly installed, secured, and ready for use. MySQL handles the data persistence for web applications, storing dynamic content and interacting with PHP scripts that query the database.As the database engine for the LAMP stack, MySQL ensures that web applications can handle complex queries, store user data, and interact with the front-end seamlessly.

4.5. Linux / Ubuntu

Here comes another big Operating system in our project, Linux/Ubuntu which serves as the base operating system for the EC2 instances in this project. Ubuntu is a popular choice for cloud-based server environments due to its stability, security, and extensive community support. It provides the environment needed for installing and running the LAMP stack.The Ubuntu server is pre-configured with the necessary dependencies for installing Apache, MySQL, and PHP. Additionally, Ubuntu allows seamless integration with Ansible, which manages configuration and ensures that the services are properly installed and running.Using Ubuntu also ensures that the project leverages an open-source environment with wide community support, making it an ideal choice for cost-effective and reliable cloud deployments.

4.6. Apache

Apache is the widely used web server that serves the PHP applications hosted on the EC2 instance. As part of the LAMP stack, Apache is configured using Ansible to handle HTTP requests, serve static files, and process PHP scripts.The Ansible playbook ensures that Apache is installed, started, and configured to allow PHP execution. Apache serves as the interface between the web browser and the server, handling requests and returning responses.Apache is one of the most widely used web servers, and its integration with PHP ensures that dynamic web content can be served effectively on the EC2 instances.

4.7. PHP

PHP is the scripting language that processes dynamic content (Dynamic Webpages) on the web server. It allows the server to execute scripts and generate dynamic web pages based on data from the MySQL database. In this project, Ansible is used to install PHP and its necessary extensions (like php-mysql) on the EC2 instance. PHP-MySQL Extension is a module that allows PHP to communicate with MySQL databases. It enables PHP scripts to interact with MySQL databases by executing queries, retrieving results, and performing other database-related tasks. The extension is essential for creating dynamic, data-driven websites or applications where PHP is used as the server-side language, and MySQL handles the data storage.

PHP interacts with Apache to process dynamic content, such as handling user input, displaying database-driven data, and managing session states for web applications. With PHP installed and properly configured, the EC2 instance is ready to serve dynamic web content as part of the LAMP stack.

4.8 VS Code / Terminal

VSCode (Visual Studio Code) is a free, open-source code editor used for writing and editing programming code. It supports multiple languages, features debugging tools, and extensions to enhance productivity for developers. VS Code is used for editing configuration files, playbooks, and other scripts, providing syntax highlighting, linting, and extensions to help manage code.Terminal is used to execute the commands that apply Terraform configurations, run Ansible playbooks, and interact with AWS. It is essential for testing and troubleshooting the scripts, as well as for verifying that the infrastructure and software configurations are running as expected.

4.9 Integration of Tools:

These tools work together seamlessly to automate the entire process of provisioning and configuring the LAMP stack on AWS EC2 instances. Terraform handles the infrastructure provisioning, ensuring that the necessary resources, such as EC2 instances, VPC, and security groups, are set up correctly. Once the infrastructure is ready, Ansible takes over, automating the installation and configuration of the LAMP stack components Apache, MySQL, and PHP.

AWS provides the cloud infrastructure, offering scalable and reliable resources to host the web application. MySQL serves as the database, Apache is the web server handling client requests, and PHP processes dynamic content, all forming the core components of the backend and frontend.

By leveraging VSCode as the development environment, coding, testing, and debugging become more efficient. Linux (Ubuntu) runs the server, while SQL manages data queries and interactions. Together, these tools enable a highly automated, reliable, scalable, and cost-efficient LAMP stack deployment.

Chapter 5 : Terraform Implementation

Terraform plays a central role in automating the infrastructure provisioning for this project. As an Infrastructure as Code (IAC) tool, Terraform allows us to define the cloud infrastructure in configuration files using HashiCorp Configuration Language (HCL). These configuration files automate the creation of resources such as EC2 instances, security groups, and networking components on AWS. Terraform Configuration Files: The main Terraform configuration consists of several files, each serving a different purpose. The main.tf file defines the AWS provider and the necessary resources, including the EC2 instances, security groups, and output values.

```
up2290345@up2290345-VirtualBox: ~
Connecting to releases.hashicorp.com (releases.hashicorp.com)|143.204.68.86|:443
... connected.
HTTP request sent, awaiting response... 200 OK
Length: 26744662 (26M) [application/zip]
Saving to: ‘terraform_1.8.5_linux_amd64.zip’

terraform_1.8.5_lin 100%[===================>]  25.50M  17.7MB/s    in 1.4s

2025-08-18 21:18:01 (17.7 MB/s) - ‘terraform_1.8.5_linux_amd64.zip’ saved [26744
662/26744662]

up2290345@up2290345-VirtualBox:~$ unzip terraform_1.8.5_linux_amd64.zip
Archive:  terraform_1.8.5_linux_amd64.zip
  inflating: LICENSE.txt
  inflating: terraform
up2290345@up2290345-VirtualBox:~$ sudo mv terraform /usr/local/bin/
up2290345@up2290345-VirtualBox:~$ terraform -version
Terraform v1.8.5
on linux_amd64

Your version of Terraform is out of date! The latest version
is 1.12.2. You can update by downloading from https://www.terraform.io/downloads
.html
up2290345@up2290345-VirtualBox:~$
```

main.tf: This file contains the configuration for creating the EC2 instance, the AMI to be used, and the instance type (e.g., t3.micro). It also specifies the security group with open ports (22, 80, 443) for SSH, HTTP, and HTTPS access.

outputs.tf: This file contains output values, such as the public IP of the provisioned EC2 instance, which can be easily referenced after execution.

security_group.tf: Defines the security group to allow inbound traffic on ports 22 (SSH), 80 (HTTP), and 443 (HTTPS). The security group is set to allow traffic from all IPs (0.0.0.0/0) for testing purposes. However, this is something we will address in the security section.

5.1 Terraform Workflow and Execution:

Initialization:

First, Terraform is initialized using the "**terraform init**" command. This command sets up the environment by downloading the necessary provider plugins and preparing Terraform to apply the configurations.

Execution Plan:

The "**terraform plan**" command is used to generate an execution plan that outlines the changes Terraform will make to the infrastructure. This step is crucial to verify that the resources will be created correctly before applying them. Apply

Configuration:

The "**terraform apply**" command is used to apply the configuration, provisioning the EC2 instance, security group, and other resources. After execution, Terraform displays the instance public IP as an output. Below is an example screenshot showing the successful application of the Terraform configuration.

```
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_instance.lamp_server: Creating...
aws_instance.lamp_server: Still creating... [10s elapsed]
aws_instance.lamp_server: Creation complete after 13s [id=i-0ce0cc9acd7f96b61]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

instance_ip = "16.171.65.27"
sriram@sriram-VirtualBox:~/lamp-auto-deploy$
```

5.2 Error Handling and Troubleshooting:

During the implementation of Terraform, several **errors** were encountered, which required troubleshooting and adjustments. One of the key challenges involved selecting the correct **instance type**. Initially, an **invalid instance type** (e.g., t2.micro) was selected, which caused issues during provisioning. Terraform indicated that the selected instance type was not eligible for the **AWS Free Tier**. After researching the issue, the instance type was switched to **t3.micro**, which is eligible for the AWS Free Tier and resolved the issue. The error message provided by Terraform helped identify the problem and led to the change in the instance type.

Additionally, several errors related to permissions and resource conflicts were identified during the initial application of the configuration. Terraform provided clear error messages indicating missing permissions or existing resource conflicts, allowing for quick resolution.

5.3 Security Issues:

In terms of security, one of the critical aspects of the Terraform configuration was the security group. Initially, the security group was set to allow all incoming traffic on ports 22, 80, and 443 using "**0.0.0.0/0"** for the CIDR block which is good for testing in multiple machines but this allowed public access to the EC2 instance from any IP address, which is not ideal for production environments.

To tighten the security, it is important to implement more restrictive CIDR blocks for inbound traffic. For example, SSH (port 22) should only be accessible from specific IP addresses (developer's public IP), while HTTP (port 80) and HTTPS (port 443) should be allowed for general public access to the web server. Implementing private IPs and using Virtual Private Clouds (VPCs) can further enhance the security of the EC2 instances and applications hosted on AWS. By restricting access to only necessary ports and IP addresses, we significantly reduce the potential attack surface for malicious actors.The security group configuration was updated, and once the new CIDR block settings were applied, Terraform was re-executed successfully, ensuring both functionality and security.

5.4 Terraform's Role in the Project:

Terraform serves as the **foundation** of this project, enabling us to automate the provisioning of cloud infrastructure. By using Terraform's **declarative configuration** files, we can describe the desired state of the infrastructure and automatically apply the changes, ensuring consistency across different environments. Terraform also provides **idempotency**, which means that we can apply the same configuration multiple times without affecting the environment if no changes are detected.

Terraform is tightly integrated with **AWS** and forms the backbone of the project's **automation process**. It ensures that **reproducibility** is achieved, allowing us to easily recreate the infrastructure in different AWS accounts or regions by simply running "**terraform apply**".

5.5 Conclusion:

The Terraform implementation in this project plays a key role in ensuring that the LAMP stack infrastructure is provisioned in a consistent and automated manner. Through the use of configuration files, Terraform makes it possible to automate the creation of EC2 instances, security groups, and other resources on AWS. The tool's integration with Ansible completes the automation process, enabling us to automate both infrastructure provisioning and application configuration, ultimately creating a fully automated and scalable deployment solution.

Chapter 6: Ansible Implementation

Ansible is a widely-used **open-source automation tool** that plays a vital role in this project for **configuration management**. While **Terraform** is used to provision the infrastructure (such as **EC2 instances**, **VPCs**, and **security groups**), **Ansible** is responsible for configuring the server environment by **installing the LAMP stack** (Linux, Apache, MySQL, PHP) and managing its configuration. The key objective of using **Ansible** in this project is to automate the configuration of the EC2 instance created by Terraform, ensuring that the necessary software and services are installed and properly configured without any manual intervention.

6.1 Introduction

Ansible is a **configuration management** and **orchestration tool** that automates tasks such as:

- **Software installation** (e.g., Apache, MySQL, PHP)
- **System configuration** (e.g., managing services, editing configuration files)
- **Task automation** (e.g., running commands or scripts remotely)

Ansible uses a declarative approach, meaning that you define the desired state of your infrastructure, and Ansible takes care of making sure that the system reaches that state. The system is **idempotent**, meaning that we stated earlier running the same playbook multiple times will result in the same outcome without causing any undesired changes.

The central concept in Ansible is its **playbook**, which is written in **YAML** and describes the tasks that need to be performed on one or more remote machines (in this case, EC2 instances). Each task can involve installing software, configuring settings, or executing commands.

6.2 Ansible Inventory and Playbooks:

Inventory File:

Before Ansible can manage the EC2 instances, we need an inventory file. This file contains the list of EC2 instances that Ansible will connect to and manage. The inventory file can be manually created or dynamically generated using Terraform output.

The Inventory file consists of the EC2 instance's public IP, the current SSH user and the Private key which is used for authentication. Ansible supports dynamic inventory as well, where the inventory file is generated dynamically based on the output from Terraform. For example, Terraform can output the EC2 instance's IP address and provide it to Ansible automatically.

Playbook:

As this is also a file but we call this as a playbook to say in brief Ansible Playbook as this is the file which automates our LAMP stack so this is called a playbook. The playbook contains multiple tasks, each one defining a specific action to be performed on the remote EC2 instance. These tasks include installing Apache, MySQL, and PHP, configuring the Apache server to serve PHP files, and ensuring that the services are started and enabled to run on boot; these tasks can be altered to user’s requirements.

Explanation of the Playbook:

In Ansible, a **Playbook** is a YAML file used to automate the configuration and management of systems. It contains a set of tasks that are executed on remote servers (in this case, **EC2 instances**) to perform specific actions. The **LAMP stack playbook** is a crucial component of the automation process in this project. It simplifies the deployment and configuration of the LAMP stack, which includes **Apache**, **MySQL**, and **PHP**, by automating the installation and setup on the EC2 instance.

Each task in the playbook is designed to perform a specific action, such as installing a software package, configuring a service, or setting up a file. For example, the playbook would include tasks like installing **Apache** with a specific version, configuring the **MySQL** server, and ensuring **PHP** is installed along with the necessary modules. The playbook also ensures that the **Apache server** is configured to serve **PHP** files and that the necessary services (Apache, MySQL, PHP) are enabled to start automatically on system boot.One of the key benefits of an Ansible playbook is its **idempotence**. This means that if the playbook is executed multiple times, it will produce the same result each time, without causing any unintended changes or errors. This is particularly useful when replicating the setup in different environments or AWS accounts. The playbook can be reused across different EC2 instances, ensuring that the LAMP stack is consistently and reliably deployed.

Additionally, the playbook is highly **customizable**. Users can modify it to fit their specific needs, such as configuring custom ports, modifying Apache settings, or choosing different versions of MySQL and PHP. As a result, the playbook can be adapted to various use cases, making it a flexible tool for developers and system administrators.Once the playbook is executed, Ansible ensures that the LAMP stack is fully set up and running on the EC2 instance, ready for hosting dynamic web applications. By automating these processes, the playbook saves time, reduces manual errors, and ensures consistency across deployments.

6.3. Running the Ansible Playbook:

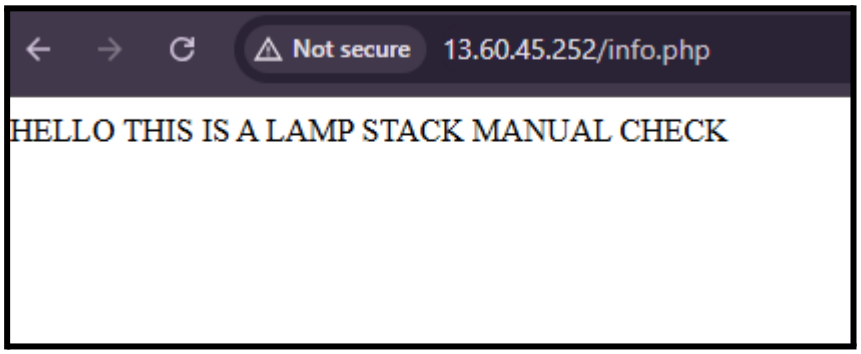
Once the inventory file and playbook are ready, the playbook can be executed by the Command, “**ansible-playbook -i inventory lamp.yml**”

```
sriram@sriram-VirtualBox: ~/lamp-auto-deploy
sriram@sriram-VirtualBox: ~/lamp-auto-deploy        sriram@sriram-VirtualBox: ~

sriram@sriram-VirtualBox:~/lamp-auto-deploy$ vim lamp.yml
sriram@sriram-VirtualBox:~/lamp-auto-deploy$ vim lamp.yml
sriram@sriram-VirtualBox:~/lamp-auto-deploy$ ansible-playbook -i hosts lamp.yml

PLAY [lamp_servers] ************************************************************************************

TASK [Gathering Facts] *********************************************************************************
[WARNING]: Platform linux on host 56.228.31.120 is using the discovered Python interpreter at
/usr/bin/python3.7, but future installation of another Python interpreter could change the meaning of that path.
See https://docs.ansible.com/ansible-core/2.16/reference_appendices/interpreter_discovery.html for more
information.
ok: [56.228.31.120]

TASK [Update all packages] *****************************************************************************
ok: [56.228.31.120]

TASK [Update all packages (Ubuntu)] ********************************************************************
skipping: [56.228.31.120]

TASK [Install Apache] **********************************************************************************
ok: [56.228.31.120]

TASK [Install Apache (Ubuntu)] *************************************************************************
skipping: [56.228.31.120]

TASK [Start and enable Apache] *************************************************************************
ok: [56.228.31.120]

TASK [Install MySQL / MariaDB] *************************************************************************
ok: [56.228.31.120]

TASK [Install MySQL (Ubuntu)] **************************************************************************
skipping: [56.228.31.120]

TASK [Start and enable MySQL] **************************************************************************
ok: [56.228.31.120]

TASK [Install PHP] *************************************************************************************
changed: [56.228.31.120]

TASK [Install PHP (Ubuntu)] ****************************************************************************
skipping: [56.228.31.120]

TASK [Create test PHP page] ****************************************************************************
changed: [56.228.31.120]

PLAY RECAP *********************************************************************************************
56.228.31.120              : ok=8    changed=2    unreachable=0    failed=0    skipped=4    rescued=0    ignored=
0
```

As this command tells Ansible to use the **inventory** file and run the playbook on the EC2 instance, automatically installing and configuring the LAMP stack.

Expected Output:

Once the playbook finishes running successfully, Apache should be installed and running on the EC2 instance, MySQL should be installed and secured, and PHP should be ready to serve dynamic content. You can verify this by accessing the **public IP** of the EC2 instance in a web browser.



6.4 **Error Handling and Troubleshooting:**

During the execution of the **Ansible playbook**, several challenges were faced. The most common issue was ensuring that the correct packages were installed, especially when dealing with different versions of software like **PHP** and **MySQL**. For example, when I installed PHP and MySQL extensions often have version mismatches, and the system might attempt to install incompatible versions. To troubleshoot, we relied on **verbose logging** in Ansible by adding the flag for more detailed output, which helped identify specific errors, such as missing dependencies or incorrect package names. Additionally, the **idempotent nature** of Ansible allowed us to rerun the playbook without causing any side effects, ensuring that changes were applied consistently.

6.5 Ansible's Role in the Project:

Ansible plays a pivotal role in the automation of the LAMP stack installation and configuration, which is a critical aspect of this project. While **Terraform** handles the provisioning of infrastructure, Ansible takes it a step further by ensuring that the **EC2 instances** provisioned by Terraform are not only created but also fully functional and ready to serve dynamic web content. Ansible automates the installation of key software components, including **Apache**, **MySQL**, and **PHP**,

ensuring that the LAMP stack is set up efficiently and consistently across all environments.

By using Ansible, the entire process of software setup is streamlined and reducing the potential for human error and eliminating the need for manual intervention. This automation significantly accelerates the deployment process, allowing for quicker rollouts and reducing downtime. Furthermore, Ansible's idempotent nature ensures that running the same playbook multiple times won't cause unintended changes or disruptions, which is especially important for maintaining stability in production environments.

When combined with **Terraform**, Ansible creates a powerful, **robust, and automated deployment pipeline** that guarantees the infrastructure and software configurations are consistently applied. This integration ensures that the setup is repeatable and scalable, which is particularly advantageous in **development, staging**, and **production environments**. The ability to automatically replicate the setup across different AWS accounts or instances ensures that all deployments are identical, improving reliability and consistency, which are essential in high-performance web hosting.

6.6 Conclusion:

The integration of **Ansible** in this project plays a critical role in automating the installation and configuration of the LAMP stack on **AWS EC2 instances**, ensuring smooth operation with minimal manual intervention. By combining **Terraform** for infrastructure provisioning and **Ansible** for configuration management, the project achieves a seamless automation process for both cloud infrastructure and software setup. This combination provides a **scalable** and **reproducible** solution for deploying web applications, making it easier to replicate setups across multiple environments. The automated approach significantly reduces human error, speeds up deployment times, and ensures consistency across different stages of development. This integration streamlines the overall deployment process, enhancing efficiency, reliability, and flexibility, making it an ideal solution for modern cloud-based application hosting.

Chapter 7: Integration and Results

The integration of Terraform and Ansible serves as the backbone of the automation in this project, streamlining the process of deploying and configuring the LAMP stack on AWS EC2 instances. These two powerful tools work in tandem to ensure that the setup is not only automated but also repeatable and scalable across different environments. Terraform provides the infrastructure, while Ansible handles the configuration of the LAMP stack, making the process efficient and consistent. During the integration, we faced challenges such as ensuring seamless communication between the two tools and resolving configuration issues. However, after overcoming these hurdles, the integration resulted in a fully automated deployment pipeline, reducing manual intervention and improving deployment times. This setup is now capable of scaling effortlessly, ensuring reliable and consistent web hosting across various AWS instances.

7.1 **Integration of Terraform and Ansible**

The integration of Terraform and Ansible allows for a full automation pipeline that covers both infrastructure provisioning and software configuration. In our project, we leverage both tools to automate the entire LAMP stack deployment process, ensuring that all components are provisioned, configured, and functional with minimal manual intervention.

Terraform Provisioning:

The process begins with Terraform, which is responsible for provisioning the underlying infrastructure. Terraform creates the AWS EC2 instance and other necessary resources such as security groups and Elastic IPs. The main.tf file defines the infrastructure, and the command ‘terraform apply’ is used to provision the resources. Terraform ensures that the infrastructure is correctly created according to the specified configuration.

Ansible Configuration:

Once the infrastructure is provisioned, Ansible takes over. The Ansible playbook is executed to configure the EC2 instance. Ansible installs the required LAMP stack components (Apache, MySQL, PHP), ensuring that the server is ready to serve dynamic web content.

Integration Flow:

- Terraform provides the EC2 instance and outputs the instance's public IP.
- This IP is then fed into the Ansible inventory file, which allows Ansible to connect to the EC2 instance and begin configuring the software.
- Ansible installs Apache, MySQL, and PHP, and configures the services to run automatically upon instance startup.
- Finally, Apache serves the PHP pages, and the MySQL database is available for data storage.

7.2 **Challenges Faced During Integration**

While integrating **Terraform** and **Ansible**, several challenges were encountered:

- **Inconsistent Configuration**: One of the challenges was ensuring that the configuration in **Terraform** (for provisioning resources) aligned perfectly with **Ansible's** requirements (for configuring the LAMP stack). Any minor discrepancies in configurations could lead to deployment failures or misconfigurations.

- **Timing and Dependencies**: The order of operations in **Terraform** and **Ansible** is critical. For example, the EC2 instance needs to be up and running before Ansible can configure it. Initially, there were some timing issues where Terraform provisioned resources, but Ansible couldn't connect because the EC2 instance wasn't fully

- **Security Configurations**: Configuring the **security groups** in **AWS** to allow the correct ports (22 for SSH, 80 for HTTP, and 443 for HTTPS) was initially a point of confusion. Ensuring that the EC2 instance was accessible without compromising security was a challenge that was carefully addressed in the Terraform configuration.

- **Dynamic Inventory**: Initially, the Ansible inventory file was static, requiring manual updates with the public IP. Later, a more dynamic approach was implemented, using Terraform output to generate the inventory file automatically, ensuring that the IP addresses were always up-to-date.

7.3 **Results of Integration**

After completing the integration of **Terraform** and **Ansible**, we successfully automated the full deployment of a **LAMP stack** on AWS. Below are the key results:

1. **Infrastructure Provisioning**:
   Using Terraform, we were able to **provision an EC2 instance** on **AWS** in a repeatable and automated manner. The EC2 instance was set up with the appropriate **AMI** (Ubuntu), **instance type** ( ), and associated security groups for secure SSH, HTTP, and HTTPS access.

2. **Automated LAMP Stack Configuration**:
   With **Ansible**, we automated the **installation and configuration of the LAMP stack** (Apache, MySQL, and PHP) on the EC2 instance. This eliminated the need for manual installation steps and ensured that the server was configured consistently every time.

3. **Scalable and Reproducible Setup**:
   The combination of Terraform and Ansible ensures that the **LAMP stack setup is scalable and reproducible**. If needed, additional instances can be provisioned using the same Terraform configuration and automatically configured using the Ansible playbook.

4. **Output Validation**:
   The EC2 instance is running **Apache**, **MySQL**, and **PHP** successfully. By accessing the EC2 instance's **public IP address** in a web browser, we can confirm that the **Apache server** serves PHP pages and the **MySQL database** is functional.

5. **Automation in Action**:
   With Terraform and Ansible integrated, the entire process from **provisioning** to **configuration** is fully automated. This not only saves time but also ensures that the setup is **consistent** and **error-free**.

7.4 Testing and Verification

During the final stages of integration, testing was conducted to verify the entire automation process. The following tests were performed:

- **Re-running Terraform and Ansible**: This was done to ensure **idempotency**, meaning that running the configuration multiple times produced the same result without errors.

- **Manual Verification**: Once the infrastructure was provisioned and the LAMP stack was configured, we manually verified that **Apache**, **MySQL**, and **PHP** were working by accessing the EC2 instance's public IP through a web browser.

- **Log Checking**: We reviewed the **system logs** (Apache and MySQL) to ensure that services were running without issues and that no errors occurred during the automation process.

7.5 Conclusion:

The integration of Terraform and Ansible has been instrumental in automating the entire process of deploying and configuring the LAMP stack on AWS EC2 instances. This powerful combination has resulted in the creation of a fully functional, scalable, and reliable web server that can be quickly replicated and extended across different environments, whether in development, staging, or production. The leveraging Terraform for infrastructure provisioning and Ansible for configuration management, we have established an efficient, automated workflow that ensures consistency and reliability across multiple deployments.

This integration eliminates the need for manual intervention in both provisioning and configuration, significantly reducing the chances of human error and improving overall deployment speed. The use of Terraform allows for easy scaling and management of cloud resources, while Ansible ensures the configuration of essential software components like Apache, MySQL, and PHP is handled seamlessly and consistently. The ability to reuse and replicate this setup across various environments ensures a streamlined process for future deployments, making it both efficient and scalable.

Overall, the combination of Terraform and Ansible exemplifies best practices in DevOps by enabling a highly automated, repeatable, and scalable infrastructure setup. This approach not only simplifies the process of managing cloud-based environments but also enhances the reliability and maintainability of the web hosting infrastructure, making it an ideal solution for modern, cloud-native application deployments.

Chapter 8: Challenges and Learnings

This project has been a journey of **continuous problem-solving** and **learning**. Throughout the course of the project, several challenges arose, each prompting me to ask critical questions that ultimately guided me toward the solutions. Below, I will detail the **challenges faced** during the project's implementation, the **questions I raised** during each challenge, and the **learnings** that resulted from those challenges.

8.1 Choosing the Correct EC2 Instance Type

Challenge:

The project started with the decision to choose an appropriate **instance type** for the AWS EC2 instance. Initially, I selected a **t2.micro** instance, but it resulted in errors because the selected instance type wasn't eligible for the AWS Free Tier. The **error message** provided during Terraform execution indicated that the instance type was incompatible with the Free Tier, causing the project to come to a halt.

Questions Raised:

- **Why is my instance type incompatible with the AWS Free Tier?**
- **Which instance types are eligible for the AWS Free Tier?**
- **How can I ensure that the instance type I select will work seamlessly with Terraform and Ansible?**

Learnings:

After investigating the issue, I discovered that the **t2.micro** instance type was not eligible under certain circumstances. I switched to a **t3.micro** instance, which was compatible with the Free Tier. This change not only resolved the error but also gave me a better understanding of AWS instance types and their Free Tier eligibility. The learning here was to validate compatibility and ensure the resources I used fit within the project constraints, particularly around cost and AWS Free Tier limits.

8.2 AWS IAM Configuration and Access Issues

Challenge:

I encountered challenges while configuring **IAM roles** and **permissions** for AWS. The **terraform apply** command initially failed due to **access control** issues. The **IAM user** I was using didn't have sufficient permissions to create and manage the necessary resources.

Questions Raised:

- **What permissions are required to use Terraform with AWS?**
- **Why am I getting permission errors when trying to provision resources?**
- **How do I properly configure IAM roles and policies to allow Terraform to create resources?**

Learnings:

Through this experience, I came to understand the critical importance of AWS Identity and Access Management (IAM) in cloud operations. I realized that ensuring the IAM user had the correct policies attached was fundamental for successful deployment, particularly permissions related to managing EC2 instances and security groups. This challenge highlighted the need for careful configuration of IAM roles for both Terraform and Ansible, as they are essential components for automating cloud infrastructure and maintaining secure access. Proper IAM setup not only helps manage resources effectively but also ensures that access control is maintained, preventing unauthorized access to critical systems. As a result, this experience significantly enhanced my ability to handle access control and security within cloud environments, improving my understanding of cloud security best practices. Going forward, I became more proficient in managing IAM roles and ensuring that security considerations are integrated into the deployment process.

8.3 Installing and Configuring Tools

Challenge:

During the initial setup, I faced difficulties installing the necessary tools, such as **Terraform**, **Ansible**, and **AWS CLI**. Some tools were **incompatible** with my operating system, while others required specific versions to function correctly. I also faced issues with ensuring that all tools were **properly configured** to work together.

Questions Raised:

- **What are the correct installation steps for each tool?**
- **How do I ensure compatibility between Terraform, Ansible, and AWS CLI?**
- **How can I integrate all tools to work seamlessly on a single EC2 instance?**

Learnings:

The installation of each tool in this project required careful troubleshooting, particularly to ensure the correct versions were installed and that the tools interacted seamlessly with each other. Through this process, I learned the importance of thoroughly following each tool's official documentation and verifying their compatibility with the operating system. I also gained a deeper understanding of dependency management, which helped me resolve conflicts and ensure the proper functioning of each tool. This experience taught me valuable lessons in system setup and configuration, making it easier to replicate and troubleshoot similar environments in the future. It emphasized the need for attention to detail and a methodical approach to installing and managing software in complex environments.

8.4 Challenges in Connectivity and SSH Access

Challenge:

One of the ongoing challenges was **connecting** to the EC2 instance via **SSH**. I encountered frequent **permission denied** errors, especially when attempting to log in with the **ubuntu** user or using the **SSH key** for authentication.

Questions Raised:

- **Why is the SSH connection not being established even after provisioning the EC2 instance?**

- **What could be causing the permission issues while accessing the EC2 instance?**

- **How do I configure the EC2 instance for seamless SSH access using the key pair?**

Learnings:

Through troubleshooting, I discovered that SSH-related issues were often due to **incorrect key pair configurations** or **wrong SSH user**. I learned the correct approach for handling **SSH authentication** and gained insights into **EC2 instance access control**, ensuring that the instance was properly configured to allow SSH access using the key pair. This experience reinforced the importance of **properly configuring access** and **verifying connectivity** before proceeding with the rest of the configuration tasks.

8.5 Rebooting and Virtual Machine Management

Challenge:
Several times during the process, I had to **reboot the system** or **recreate the VM**. This was due to issues related to **tool installation failures**, **configuration errors**, or simply needing to restart from a clean state. Recreating a fresh VM meant starting over, which was time-consuming but helped solve underlying issues.

Questions Raised:

- **How do I efficiently manage the VM environment without having to restart from scratch?**

- **What backup or cloning methods can I use to preserve progress before rebooting?**

- **How can I optimize the workflow to avoid repeated reboots?**

Learnings:
Although frequent reboots were inconvenient, I learned the value of **virtual machine snapshots** and **cloning**. These methods allowed me to quickly return to a previous stable state without losing all progress. I also learned to **manage resources** more efficiently, including using **VM cloning** to set up backup environments where I could quickly test changes without affecting the main VM.

8.6 Idempotency and Re-running Ansible Playbooks

Challenge:
Another challenge was ensuring **idempotency** when re-running the **Ansible playbook**. After making changes, I needed to ensure that re-running the playbook wouldn't cause the configuration to break or lead to unintended modifications.

Questions Raised:

- **How can I ensure that Ansible playbooks are idempotent?**
- **What strategies can I use to prevent accidental reconfiguration when re-running the playbook?**

Learnings:
I learned that **Ansible's idempotent nature** is a key feature, meaning that it only makes necessary changes to the system. To fully leverage this, I ensured that the playbook was written with **correct checks** and **task dependencies** to avoid redundant operations. This experience reinforced the importance of creating **robust playbooks** and maintaining **consistent states** across deployments.

8.7 Security Issues and Configuration Management

Challenge:

The **security configuration** of the EC2 instance was another significant challenge. During the setup, the **security group** was initially configured to allow **open access** (0.0.0.0/0), which made the server publicly accessible. Although this was necessary for testing, it was **insecure** for production use.

Questions Raised:

- **How do I configure a secure firewall for the EC2 instance?**
- **What are the best practices for securing an AWS EC2 instance?**
- **How do I restrict SSH access to specific IP addresses while maintaining the functionality of HTTP and HTTPS?**

Learnings:

Through research and testing, I realized the importance of restricting access to critical services such as **SSH**. I updated the security group to limit SSH access to specific IPs and ensured that only the necessary ports (80, 443) were open to the public. This reinforced my understanding of **network security best practices** and the importance of maintaining secure cloud environments.

8.8 Conclusion:

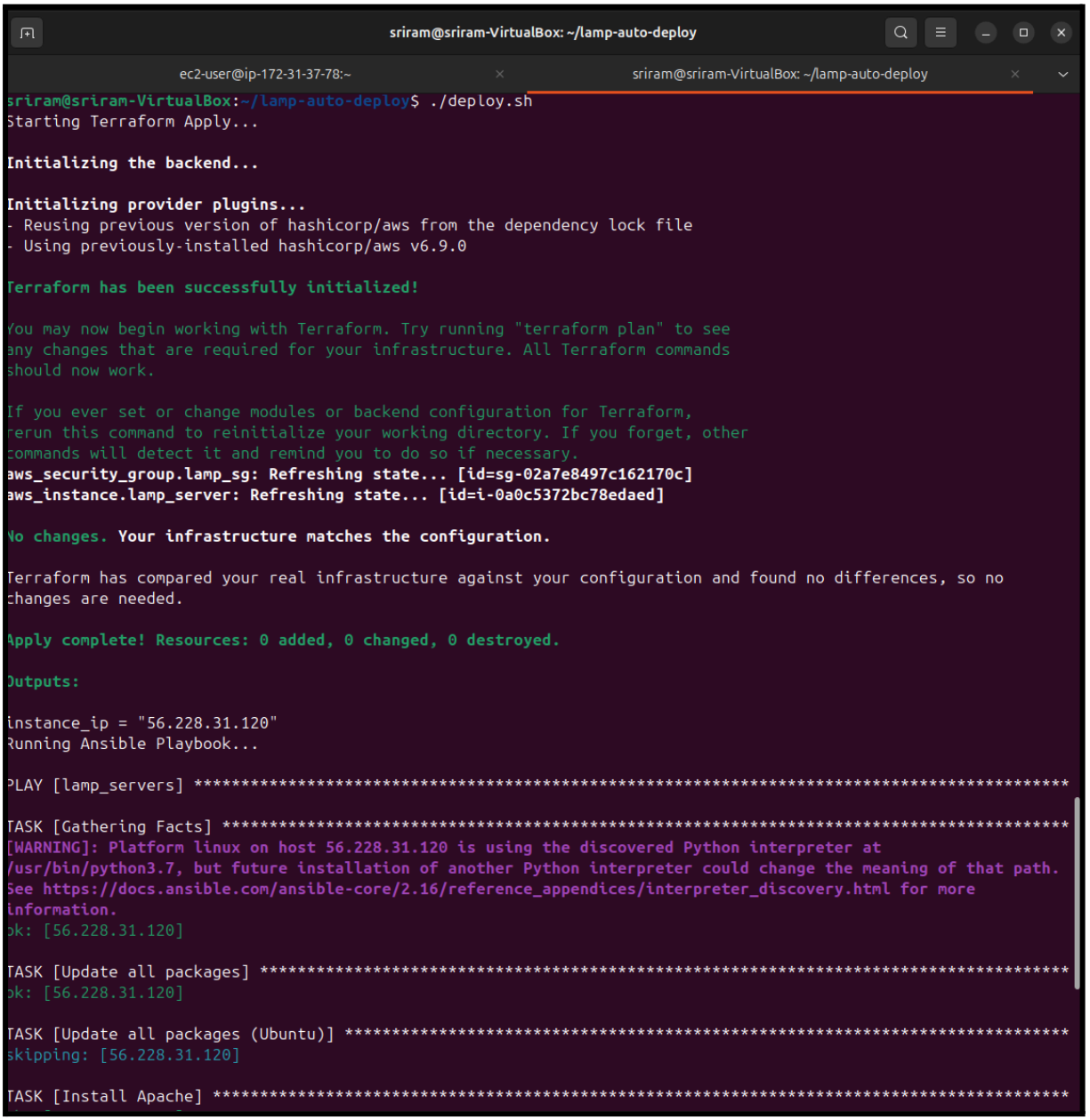
Throughout this project, the challenges faced prompted me to ask critical questions that led to valuable insights and solutions. From configuring IAM roles to managing **security** and **tool installations**, each challenge served as a learning experience that deepened my understanding of cloud infrastructure, automation tools, and best practices in DevOps. The **repeated reboots** and **frequent troubleshooting** provided opportunities to refine the process and improve overall system management. By addressing these challenges, I developed the skills necessary to automate the deployment of a **LAMP stack** on **AWS** effectively.

This project not only helped me understand the technical aspects of **Terraform**, **Ansible**, and **AWS** but also strengthened my problem-solving abilities and resilience in the face of challenges.

Chapter 9: Screenshots & Testing Evidence

9.1. Terraform Execution and EC2 Instance Provisioning

This section includes screenshots showing the execution of **Terraform commands**, like 'terraform apply' along with the **output** showing the **EC2 instance creation** process and the public IP output.



```
sriram@sriram-VirtualBox: ~/lamp-auto-deploy
ec2-user@ip-172-31-37-78:~        sriram@sriram-VirtualBox: ~/lamp-auto-deploy
sriram@sriram-VirtualBox:~/lamp-auto-deploy$ ./deploy.sh
Starting Terraform Apply...

Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.9.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
aws_security_group.lamp_sg: Refreshing state... [id=sg-02a7e8497c162170c]
aws_instance.lamp_server: Refreshing state... [id=i-0a0c5372bc78edaed]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no
changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

instance_ip = "56.228.31.120"
Running Ansible Playbook...

PLAY [lamp_servers] ***********************************************************************************

TASK [Gathering Facts] ********************************************************************************
[WARNING]: Platform linux on host 56.228.31.120 is using the discovered Python interpreter at
/usr/bin/python3.7, but future installation of another Python interpreter could change the meaning of that path.
See https://docs.ansible.com/ansible-core/2.16/reference_appendices/interpreter_discovery.html for more
information.
ok: [56.228.31.120]

TASK [Update all packages] ****************************************************************************
ok: [56.228.31.120]

TASK [Update all packages (Ubuntu)] *******************************************************************
skipping: [56.228.31.120]

TASK [Install Apache] *********************************************************************************
```

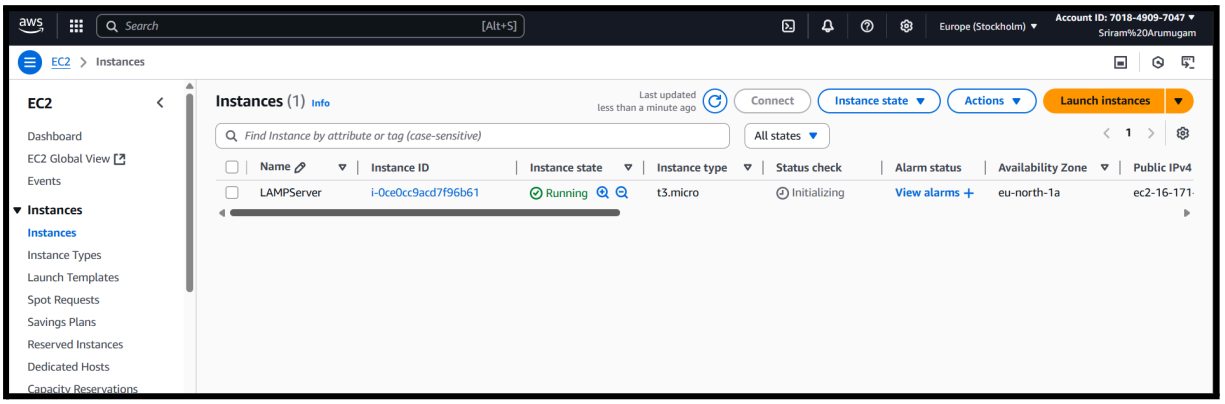
9.2. Ansible Playbook Execution

Here, include screenshots showing the execution of the **Ansible playbook** along with the corresponding **output logs** to confirm successful software installation and configuration (Apache, MySQL, PHP).

```
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_instance.lamp_server: Creating...
aws_instance.lamp_server: Still creating... [10s elapsed]
aws_instance.lamp_server: Creation complete after 13s [id=i-0ce0cc9acd7f96b61]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

instance_ip = "16.171.65.27"
sriram@sriram-VirtualBox:~/lamp-auto-deploy$
```

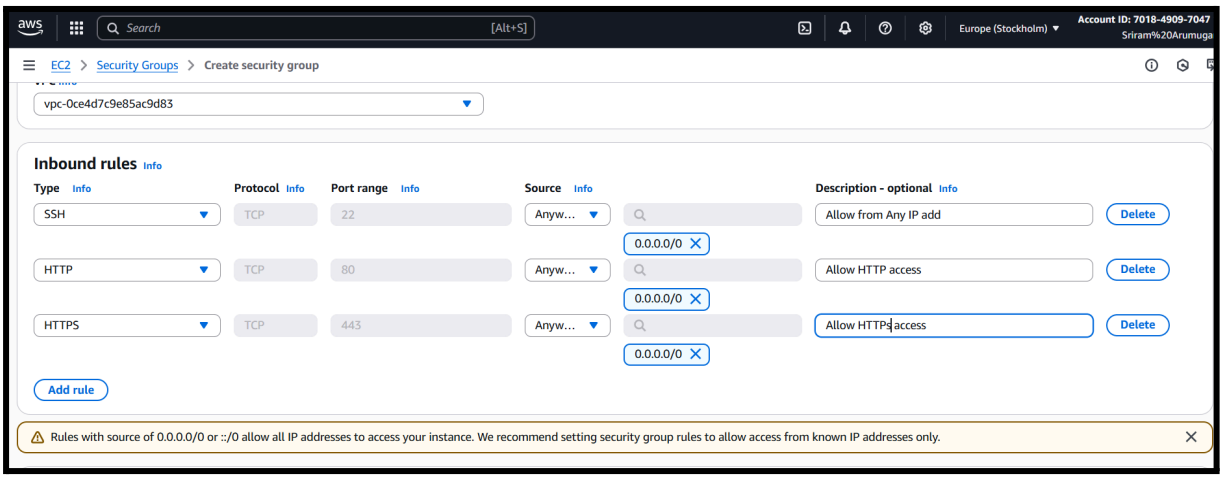
9.3 AWS EC2 Instance Page

This screenshot shows the **AWS EC2 instance** page in the **AWS Management Console**, where the instance t3.micro was provisioned using **Terraform**. The instance is running on **Ubuntu**, and this page displays the **public IP** assigned to the instance, which is used to **access** the server via **SSH** and **HTTP**



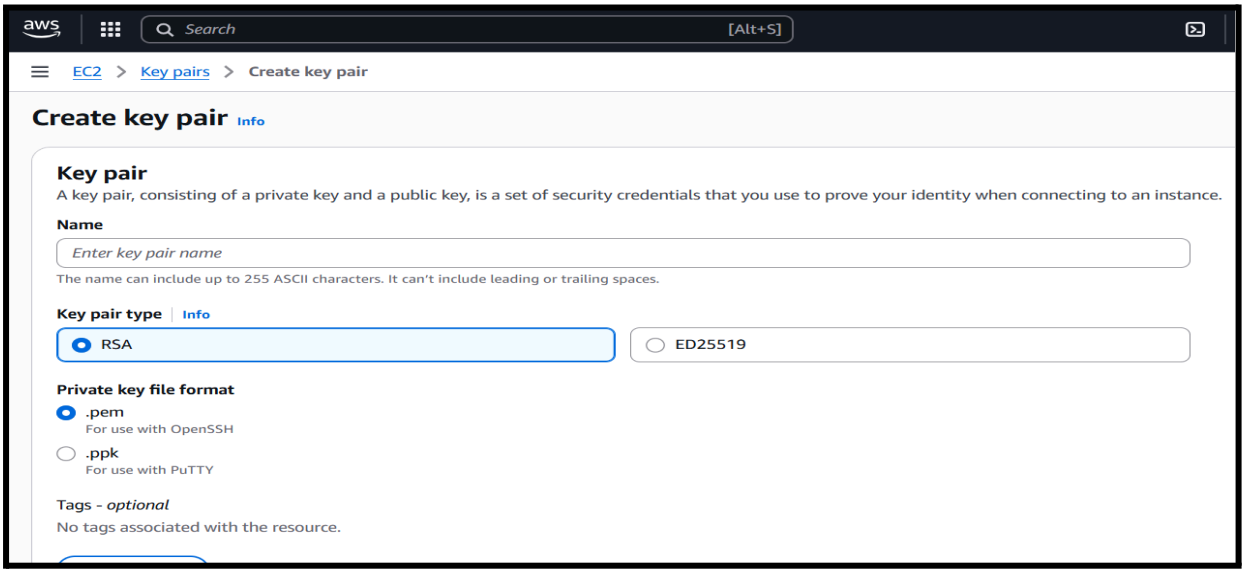
9.4 AWS Security Group Configuration

This screenshot depicts the **AWS Security Group** configuration. The **security group** was created in **Terraform** to allow **SSH access** (port 22), **HTTP** (port 80), and **HTTPS** (port 443) to the EC2 instance. This ports all open ports for testing purpose but in production environment only the necessary ports are open for accessing the server, while the rest remain **secure**



9.5 RSA Key Pair (.pem) Configuration

This screenshot shows the **RSA key pair** used for **SSH access** to the EC2 instance. The key pair was generated using **AWS IAM** and saved as a **.pem file**. The private key is used to authenticate and securely connect to the **AWS EC2 instance** during the setup and configuration process



Chapter 10 : Conclusion & Future Improvements

10.1 Conclusion

This project, **Infrastructure Automation: LAMP Stack Auto-Deployment Using Terraform and Ansible**, has successfully achieved the goal of automating the deployment of a LAMP stack on **AWS EC2** instances. By leveraging **Terraform** for infrastructure provisioning and **Ansible** for configuration management, we have created a fully automated, scalable, and reproducible deployment pipeline. This project demonstrates key principles of **DevOps**, particularly around **Infrastructure as Code (IAC)**, **automation**, and **continuous delivery**.

Throughout the project, we faced various challenges, from configuration mismatches to integration issues between the tools. However, these challenges were addressed with persistent troubleshooting and thorough research. The project also successfully showcased the value of using tools like **Terraform** and **Ansible** in a real-world cloud environment, with the automation reducing both **manual work** and **human error** in system configuration. The **idempotency** of both tools ensured that the automation was **reliable** and **consistent**, enabling the deployment of a **LAMP stack** in a repeatable manner every time.

The **final output** of the project—a fully functional LAMP stack on AWS, which demonstrates the power of **automation** and the benefits it provides in terms of **efficiency**, **scalability**, and **cost-effectiveness**. This automated deployment process is now capable of being reproduced on any EC2 instance with minimal manual intervention. The project not only meets its objectives but also serves as a **real-world example** of how **DevOps** practices can be implemented for efficient cloud infrastructure management.

10.2 Future Improvements

While the project successfully automated the deployment of the **LAMP stack**, there are several opportunities for **future improvements** that could enhance its **capabilities**, **security**, and **scalability**, particularly in the context of **industry needs**:

1. **Security Enhancements**:

    - One of the key concerns in this project was the **public accessibility** of the EC2 instance due to the security group being configured with a wide-open CIDR block (0.0.0.0/0). While this configuration is appropriate for testing, it presents a **security risk** in a production environment. Moving forward, **restricted access** should be implemented, where only specific **IP addresses** or **VPNs** can access the instance, particularly for **SSH access** (port 22).

    - Furthermore, the security configurations should be refined by leveraging **AWS Identity and Access Management (IAM)** policies to ensure that only the necessary roles and permissions are granted to the automation tools (Terraform, Ansible). This will help to prevent unauthorized access and enforce **least privilege** principles.

2. **Scalability and Load Balancing**:

    - As the project progresses, **horizontal scaling** should be introduced. This can be done by automatically deploying additional EC2 instances based on **demand** and using **Elastic Load Balancing (ELB)** to distribute traffic among multiple instances. The architecture can be modified to **auto-scale** by configuring **AWS Auto Scaling** groups, making the infrastructure **adaptive** to varying levels of load and improving its **resilience** and **high availability**.

- This would be highly beneficial for hosting **large-scale web applications** that experience unpredictable traffic spikes.

3. **CI/CD Integration**:

   - To improve the **development cycle**, the next step would be to integrate the current automation process into a **Continuous Integration/Continuous Deployment (CI/CD)** pipeline. By integrating tools such as **Jenkins** or **GitHub Actions**, the entire process can be fully automated, including code deployment, configuration updates, and testing. This would enhance the project's **efficiency**, **speed**, and **consistency** in delivering updates to the deployed web application.

   - This approach would also facilitate the automatic configuration of additional web services or deployment of web applications as soon as new code is committed.

4. **Cost Optimization**:

   - While the project makes use of AWS's **Free Tier**, cost optimization in cloud environments is a critical area for improvement. Future work should focus on selecting **cost-effective instance types**, **right-sizing** instances, and utilizing **AWS Spot Instances** to reduce costs further. Implementing a cost-tracking tool or integrating the solution with **AWS CloudWatch** could help monitor the costs in real-time and adjust resource allocation dynamically to avoid wastage.

5. **Multi-cloud and Hybrid Cloud Deployment**:

    - Expanding beyond AWS, future work could involve adapting the automation process to **multi-cloud** environments, incorporating other cloud providers such as **Google Cloud Platform (GCP)** or **Microsoft Azure**. This would enhance the project's flexibility and allow it to meet the diverse needs of organizations operating in **multi-cloud** ecosystems. Additionally, a **hybrid cloud** approach could be explored, where sensitive data is kept on private clouds while public cloud services are used for scaling and application delivery.

6. **Containerization and Kubernetes**:

    - To further enhance the **portability** and **scalability** of the application, the next step could involve **containerizing** the LAMP stack using **Docker** and deploying it to **Kubernetes** clusters. Containerization would allow for **environment consistency** across different development, staging, and production environments, enabling **rapid scaling** and deployment with **Kubernetes**.

10.3 Conclusion

In conclusion, this project demonstrates the power of automation using **Terraform** and **Ansible** to create a fully automated and scalable **LAMP stack** deployment on **AWS EC2 instances**. The automation significantly reduces manual effort, minimizes errors, and provides a **consistent** and **reproducible** deployment pipeline. The integration of these tools reflects **best practices** in **DevOps**, ensuring efficient cloud resource management and configuration automation.

The **future improvements** highlighted in this report, particularly around **security**, **scalability**, and **CI/CD integration**, will further enhance the project, making it more **robust**, **secure**, and **industry-ready**. As cloud technologies continue to evolve, integrating **new tools** and **best practices** into the project will ensure that it remains relevant and adaptable to the changing needs of the industry.

By leveraging **Infrastructure as Code (IAC)** and **automated configuration management**, the project not only meets its goals but also serves as a foundation for future innovations in cloud automation and DevOps practices.

References

1. **Abbas Mehdi**, "Terraform: Streamlining Infrastructure Deployment and Management through Infrastructure as Code," IEEE Explore.

2. **Milandeep Kour Bali**, "Enhancing Efficiency through Infrastructure Automation: An In-depth Analysis of Infrastructure as Code (IaC) Tools," IEEE Explore.

3. **Souradeep Banerjee**, "Infrastructure as a Code (IaC) to Software Defined Infrastructure using Azure Resource Manager (ARM)," IEEE.

4. **Erika Heidi**, "How to Use Ansible to Install and Set Up LAMP on Ubuntu 18.04," Digital Ocean.

5. **Rajat Saini**, "Evaluating the Role of Secure Shell (SSH) in Securing Wireless Networks," 2023 IEEE.

6. **HashiCorp**, “Terraform — Documentation,” HashiCorp.

7. **DigitalOcean**, “How to use Ansible with Terraform for configuration management,” Nov. 15, 2021.

8. **DigitalOcean**, “How to use Ansible to install and set up LAMP on Ubuntu,” Dec. 17, 2019.

9. **P. Singh**, “ansible-lamp-stack-automation,” GitHub repository.

10. **J. Martinez**, “Building a multi-tier LAMP stack on AWS with Terraform and Ansible,” Medium, Jun. 26, 2025.

11. **J. Martinez**, “Deploying a LAMP stack on AWS with Terraform and Ansible (Prototype to Production),” Medium, Jun. 23, 2025.

12. **D. B. Karki**, “Deploy LAMP stack website via Ansible and Terraform on AWS Cloud,” Medium, Aug. 18, 2023.

13. **S. Rashmina**, “Automating infrastructure: Combining Terraform with Ansible for provisioning and configuration,” Medium.

14. **A. Amusengeri**, “Simplified infrastructure as code with Terraform, Ansible, and AWS,” Medium.

15. **F. Dinu**, “Using Terraform & Ansible together,” Spacelift Blog.

Appendix:

Ethical Approval Status

As of the submission of this report, the **Ethical Approval Form** has not yet been received. However, the approval is pending and is available in my email. The **Ethical Approval Number** is **TETHIC-2025-111459**. I will update the report with the signed approval form once it is received

……..X……..END OF REPORT ……..x…….